

Wireshark Monitor Mode Guide for Penetration Testing

Table of Contents

1. Introduction
2. Understanding Monitor Mode
3. Prerequisites
4. Setting Up Monitor Mode
5. Capturing Wireless Traffic
6. Penetration Testing Scenarios
7. Filtering and Analysis Techniques
8. Advanced Techniques
9. Legal and Ethical Considerations
10. Troubleshooting

Introduction

This guide provides comprehensive instructions on using Wireshark in monitor mode for wireless penetration testing. Monitor mode allows you to capture all wireless traffic in range, not just traffic directed to your machine, making it essential for security assessments.

Purpose: Educational resource for authorized penetration testers, security researchers, and network administrators conducting legitimate security assessments.

Understanding Monitor Mode

What is Monitor Mode?

Monitor mode (also called RFMON - Radio Frequency Monitoring) is a special mode for wireless network interfaces that allows them to capture all 802.11 wireless traffic in range, including:

- ? Management frames (beacons, probe requests/responses, authentication, association)
- ? Control frames (RTS, CTS, ACK)
- ? Data frames (encrypted or unencrypted traffic)

Monitor Mode vs. Promiscuous Mode

Feature	Monitor Mode	Promiscuous Mode
Captures all wireless traffic	Yes	No (only associated network)
Captures management frames	Yes	Limited
Requires association	No	Yes
Channel hopping	Yes	No
Works on wireless	Yes	Yes
Works on wired	No	Yes

Prerequisites

Hardware Requirements

Supported Wireless Adapters:

- ? Atheros chipsets (AR9271, AR9287)
- ? Realtek chipsets (RTL8187, RTL8812AU)
- ? Ralink chipsets (RT2800, RT3070, RT5370)
- ? Intel adapters (limited support)

Recommended Adapters for Pentesting:

- ? Alfa AWUS036ACH (dual-band)
- ? Alfa AWUS036NH
- ? TP-Link TL-WN722N v1
- ? Panda PAU09

Software Requirements

- # Essential tools
 - Wireshark (latest version)
 - aircrack-ng suite
 - iw or iwconfig
 - Linux OS (Kali Linux, Parrot OS, or Ubuntu)

Checking Adapter Compatibility

- # Check if your adapter is detected
 - `iwconfig`
- # Check for monitor mode support
 - `iw list | grep -A 10 "Supported interface modes"`
- # Look for "monitor" in the output

Setting Up Monitor Mode

Method 1: Using airmon-ng (Recommended)

- # 1. Check current wireless interfaces
 - `iwconfig`
- # 2. Kill interfering processes
 - `sudo airmon-ng check kill`
- # 3. Enable monitor mode (replace wlan0 with your interface)
 - `sudo airmon-ng start wlan0`
- # Your interface name will change to wlan0mon or similar
- # Verify monitor mode
 - `iwconfig`

Method 2: Using iw Commands

```
# 1. Bring interface down
sudo ip link set wlan0 down
# 2. Set monitor mode
sudo iw dev wlan0 set type monitor
# 3. Bring interface up
sudo ip link set wlan0 up
# 4. Verify
iwconfig wlan0
```

Method 3: Using iwconfig (Legacy)

```
# 1. Bring interface down
sudo ifconfig wlan0 down
# 2. Set monitor mode
sudo iwconfig wlan0 mode monitor
# 3. Bring interface up
sudo ifconfig wlan0 up
# 4. Verify
iwconfig wlan0
```

Setting a Specific Channel

```
# Set to channel 6 (2.4GHz)
sudo iw dev wlan0mon set channel 6
# Set to channel 36 (5GHz)
sudo iw dev wlan0mon set channel 36
# For channel hopping, use airodump-ng (covered later)
```

Capturing Wireless Traffic

Starting Wireshark in Monitor Mode

```
# 1. Launch Wireshark with sudo privileges
sudo wireshark
# 2. Select your monitor mode interface (e.g., wlan0mon)
# 3. Click "Start capturing"
# Or from command line:
sudo wireshark -i wlan0mon -k
```

Wireshark Capture Options

Important Settings:

1. Interface: Select your monitor mode interface
2. Promiscuous Mode: Enable (though not strictly needed in monitor mode)

3. Buffer Size: Increase for high-traffic environments
4. Capture Filter: Apply if you want to limit capture scope

Using airodump-ng for Targeted Capture

```
# Scan all channels and capture to file
sudo airodump-ng wlan0mon
# Target specific channel and BSSID
sudo airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF -w capture wlan0mon
# This creates .cap files compatible with Wireshark
```

Penetration Testing Scenarios

Scenario 1: Wireless Network Discovery

Objective: Identify all wireless networks in range

```
# 1. Enable monitor mode
sudo airmon-ng start wlan0
# 2. Scan for networks
sudo airodump-ng wlan0mon
# 3. Analyze in Wireshark
sudo wireshark -i wlan0mon -k
```

What to Look For:

- ? SSIDs (network names)
- ? BSSIDs (AP MAC addresses)
- ? Channel numbers
- ? Encryption types (WEP, WPA, WPA2, WPA3)
- ? Client associations
- ? Signal strength

Wireshark Filters:

```
wlan.fc.type_subtype == 0x08    # Beacon frames
wlan.fc.type_subtype == 0x04    # Probe requests
wlan.fc.type_subtype == 0x05    # Probe responses
```

Scenario 2: Capturing WPA/WPA2 Handshakes

Objective: Capture 4-way handshake for password cracking

Step 1: Identify Target Network

```
sudo airodump-ng wlan0mon
# Note the BSSID, channel, and verify clients are connected
```

Step 2: Capture on Target Channel

```
sudo airodump-ng -c [channel] --bssid [AP_MAC] -w handshake wlan0mon
```


Step 3: Trigger Handshake (Optional)

```
# In another terminal, deauthenticate a client
sudo aireplay-ng --deauth 10 -a [AP_MAC] -c [CLIENT_MAC] wlan0mon
```

Step 4: Analyze in Wireshark

```
wireshark handshake-01.cap
```

Wireshark Filter for Handshakes:

```
eapol
```

Identifying Complete Handshake:

Look for 4 EAPOL frames (Key messages 1-4):

1. Message 1: ANonce from AP
2. Message 2: SNonce from client
3. Message 3: GTK from AP
4. Message 4: ACK from client

Verification:

```
# Verify handshake capture with aircrack-ng
aircrack-ng handshake-01.cap
```

Scenario 3: Analyzing Client Behavior

Objective: Monitor client probe requests and connections

Wireshark Filters:

```
# All probe requests
wlan.fc.type_subtype == 0x04
# Probe requests from specific device
wlan.addr == [CLIENT_MAC] && wlan.fc.type_subtype == 0x04
# Association requests
wlan.fc.type_subtype == 0x00
# Authentication frames
wlan.fc.type_subtype == 0x0b
```

What This Reveals:

- ? Previously connected networks (SSID history)
- ? Device MAC addresses
- ? Preferred networks
- ? Connection patterns
- ? Device capabilities

Scenario 4: Detecting Rogue Access Points

Objective: Identify unauthorized or malicious APs

Detection Techniques:

1. Same SSID, Different BSSID
- ```
Filter for specific SSID
```



```
wlan.ssid == "CompanyNetwork"
```

## 2. Evil Twin Detection

Look for:

- ? Multiple APs with same SSID
- ? Stronger signal than expected
- ? Different encryption methods
- ? Suspicious beacon intervals

## 3. Unusual Traffic Patterns

```
Look for deauth attacks
wlan.fc.type_subtype == 0x0c
```

# Scenario 5: Traffic Analysis and Data Extraction

Objective: Analyze unencrypted or decrypted traffic

Capturing HTTP Traffic:

```
Filter for HTTP in wireless traffic
http && wlan
```

Capturing DNS Queries:

```
dns && wlan
```

Decrypting WPA2 Traffic (if password known):

1. Edit ? Preferences ? Protocols ? IEEE 802.11
2. Enable decryption
3. Edit ? Decryption keys ? Add
  - Key type: wpa-pwd
  - Key: password:SSID

Extract Files from Decrypted Traffic:

1. File ? Export Objects ? HTTP
2. Browse extracted content

# Scenario 6: Detecting Wireless Attacks

Deauthentication Attack Detection:

```
Filter for deauth frames
wlan.fc.type_subtype == 0x0c
Count deauth frames
Statistics ? I/O Graph ? filter: wlan.fc.type_subtype == 0x0c
```

Evil Twin/MITM Detection:

```
Look for duplicate SSIDs
Statistics ? WLAN Traffic
```

WPS Attack Detection:

```
Filter for WPS frames
wps
```



# Filtering and Analysis Techniques

## Essential Wireshark Display Filters

### Basic Wireless Filters

```
All wireless traffic
wlan
Specific BSSID (AP)
wlan.bssid == aa:bb:cc:dd:ee:ff
Specific client
wlan.addr == aa:bb:cc:dd:ee:ff
Specific SSID
wlan.ssid == "NetworkName"
Specific channel (requires RadioTap)
radiotap.channel.freq == 2437 # Channel 6
```

### Frame Type Filters

```
Management frames
wlan.fc.type == 0
Control frames
wlan.fc.type == 1
Data frames
wlan.fc.type == 2
Beacon frames
wlan.fc.type_subtype == 0x08
Probe requests
wlan.fc.type_subtype == 0x04
Authentication frames
wlan.fc.type_subtype == 0x0b
Deauthentication frames
wlan.fc.type_subtype == 0x0c
```

### Encryption Filters

```
WEP encrypted data
wlan.fc.protected == 1 && wlan.wep.key
WPA/WPA2 data
wlan.fc.protected == 1
EAPOL frames (handshakes)
eapol
Unencrypted data
wlan.fc.protected == 0 && wlan.fc.type == 2
```

### Advanced Filters



```
Retry frames (possible connectivity issues)
wlan.fc.retry == 1
Fragmented frames
wlan.frag
Power saving
wlan.fc.pwrmtg == 1
To DS (client to AP)
wlan.fc.tods == 1
From DS (AP to client)
wlan.fc.fromds == 1
```

## Analysis Techniques

### 1. Network Mapping

Using Statistics:

- ? Statistics ? WLAN Traffic: Overview of all SSIDs and BSSIDs
- ? Statistics ? Endpoints: List all devices seen
- ? Statistics ? Conversations: Communication patterns

Creating a Network Map:

1. Export endpoint list
2. Note BSSID (AP) and associated clients
3. Document channels, encryption, signal strength

### 2. Timeline Analysis

```
I/O Graph for traffic patterns
Statistics ? I/O Graph
Useful filters in I/O Graph:
Graph 1: wlan.fc.type == 2 (data frames)
Graph 2: wlan.fc.type_subtype == 0x0c (deauth)
Graph 3: eapol (authentication activity)
```

### 3. Client Tracking

Following a Client:

```
Filter for specific client
wlan.addr == [CLIENT_MAC]
Follow client through connection process:
1. Probe requests
2. Authentication
3. Association
4. EAPOL handshake
5. Data transfer
```

### 4. Security Assessment





## Encryption Type Identification:

```
From beacon frames
wlan.fc.type_subtype == 0x08
Look at:
wlan.tag.number == 48 # RSN (WPA2/WPA3)
wlan.wfa.ie.wpa.version # WPA
```

## Password Attack Surface:

- ? WEP: Highly vulnerable
- ? WPA-PSK: Dictionary/brute-force attacks
- ? WPA2-PSK: Handshake capture + cracking
- ? WPA2-Enterprise: More secure, but targets credentials
- ? WPA3: Resistant to offline attacks

# Advanced Techniques

## Channel Hopping

### Using airodump-ng:

```
Hop across 2.4GHz channels
sudo airodump-ng wlan0mon
Hop specific channels
sudo airodump-ng --channel 1,6,11 wlan0mon
5GHz channels
sudo airodump-ng --band a wlan0mon
```

### Manual Channel Hopping Script:

```
#!/bin/bash
channel-hop.sh
INTERFACE="wlan0mon"
CHANNELS="1 6 11"
while true; do
 for CHANNEL in $CHANNELS; do
 iw dev $INTERFACE set channel $CHANNEL
 sleep 2
 done
done
```

## Geolocation and Wardriving

### Combining with GPS:

```
Using Kismet for GPS-tagged captures
sudo kismet -c wlan0mon
Or use airodump-ng with GPS
sudo airodump-ng --gpsd wlan0mon
```

### Wireshark with GPS Data: